

Lab 8 — Deferred Work: GPIO Interrupts and ISR-Safe Code

ESP32-S3 FIRMWARE LAB SERIES · PHASE 2 · PAST THE COURSEWORK

Estimated time	2–3 hrs
Introduces	ISR context, <code>FromISR</code> , top half / bottom half
Prerequisites	Labs 1 and 2.
Beyond the board	Nothing — the BOOT button is the interrupt source.
Target	ESP32-S3 (written against an N16R8 module: 16 MB flash, 8 MB octal PSRAM)
Framework	ESP-IDF v5.5, CMake and Ninja, driven by <code>idf.py</code>

NOTE

How to read this lab

Sections run **§8.1 objective**, **§8.2 parts and wiring**, **§8.3 how it works**, then the code, a line-by-line walkthrough, the documentation map, and verification. Callout boxes are tagged **[WARNING]**, **[KEY POINT]**, **[NOTE]**, **[DOCS]** and **[TM4C BRIDGE]**. Figures are given as captioned text and tables so every value in them is searchable and checkable.

The **TM4C bridge** notes map each topic onto the equivalent bare-metal, register-level work on a TI TM4C123 (Cortex-M4F). Skip them if that is not your background; nothing later depends on them.

8.1 Objective & TM4C bridge

Handle a hardware interrupt correctly under an RTOS: capture the event in a minimal handler and do the real work in a task. This lab maps onto a skill gap worth closing deliberately, and it is one of the densest interview topics in embedded.

TM4C BRIDGE

The bare-metal interrupt lab is: configure the timer interrupt, write the handler, clear the flag in `ICR`, enable the line in the NVIC. You already understand what an interrupt *is* at the silicon level. What is new is what an interrupt means when an **RTOS** sits underneath — an ISR can now make tasks runnable, and there is a separate set of API functions you must use to do that safely.

8.2 Parts & wiring

Item	Status
ESP32-S3 board and the onboard BOOT button (GPIO0)	onboard — nothing to wire

No external parts. If you want to see the bounce that §8.3 describes, put the analyzer on GPIO0 at 4 MHz and press the button (if you are on the expansion base, see §2.8 — GPIO0 is not brought out and you will need to hook the devkit header directly); the burst of edges a single press really produces makes the debounce discussion concrete rather than theoretical.

8.3 How it works

Polling versus interrupts — the honest comparison

In Lab 1 you read the button by **polling**: checking its level over and over in a loop. That works, and for a button it is often fine. It is worth being precise about when it stops being fine, because "interrupts are better" is a shallow answer.

	Polling	Interrupt
Latency	Up to one full loop period	Microseconds, hardware-driven
CPU cost when idle	Pays on every loop whether or not anything happened	Zero until the event occurs
Can miss short events?	Yes — a pulse shorter than the loop period is invisible	No — hardware latches the edge
Complexity	Trivial	Real: context rules, shared-data hazards
Good for	Slow, non-critical, predictable inputs	Rare or urgent events; anything with a deadline; low-power designs

The deciding question is usually "**can I afford to miss it, and can I afford to keep asking?**" A battery device that must sleep cannot poll at all — polling means staying awake — so interrupts are the only option. That is the connection to Lab 12.

What actually happens when the button is pressed

Walk the whole chain once, because "an interrupt fires" hides a lot.

1. The pin is held HIGH by the internal pull-up. You press; the switch connects it to ground and the voltage falls.
2. The GPIO peripheral is configured to watch for a **falling edge** and latches a flag in hardware the instant it sees one. This latching is why an interrupt cannot miss a brief event.
3. The interrupt controller interrupts the CPU. Whatever task was running is suspended mid-flow — its registers are saved automatically.
4. The CPU jumps to your handler. You are now in **interrupt context**: not a task, no task stack, and the scheduler is not going to run anything else until you return.
5. Your handler does its minimal work and returns. Context is restored and normal scheduling resumes — possibly into a *different* task than the one that was interrupted, if your handler woke a higher-priority one.

Step 5 is the part with no TM4C equivalent, and it is what `portYIELD_FROM_ISR()` is for.

5. Your handler does its minimal work and returns. Context is restored and normal scheduling resumes — possibly into *different* task than the one that was interrupted, if your handler woke a higher-priority one.

Step 5 is the part with no TM4C equivalent, and it is what `portYIELD_FROM_ISR()` is for.

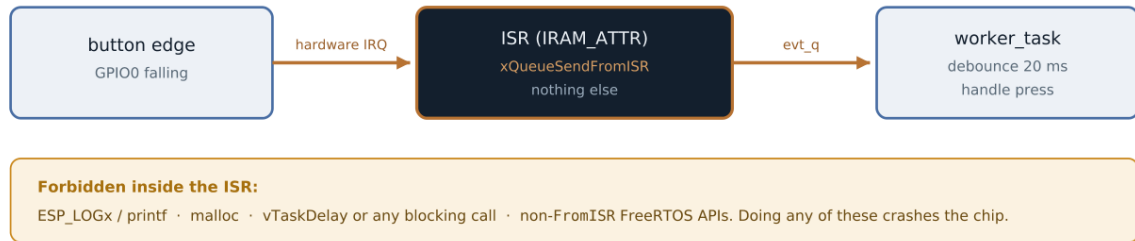


Figure 15. The top-half / bottom-half split. The ISR captures the event and returns in microseconds; a normal task does the slow work where blocking and logging are legal.

Figure 8.1. The top-half / bottom-half split. The ISR captures the event and returns in microseconds; a normal task does the slow work, where blocking and logging are legal.

Top half / bottom half interrupt handling.

Flow: button edge (GPIO falling) --hardware IRQ--> ISR marked `IRAM_ATTR`, which calls `xQueueSendFromISR` and does nothing else --evt_q--> `worker_task`, which debounces for 20 ms and then handles the press.

Forbidden inside the ISR: - `ESP_LOGx` or `printf` - `malloc` - `vTaskDelay` or any blocking call - any non-`FromISR` FreeRTOS API

Doing any of these crashes the chip.

The three ISR rules

These are the rules interviewers probe, and the reasons matter more than the rules.

Rule 1: never block. No `vTaskDelay`, no waiting on a mutex, no plain `xQueueSend` that could wait for space. Blocking means "put the caller to sleep until later" — but there is no task here to put to sleep. Attempting it corrupts the kernel's bookkeeping or crashes outright.

Rule 2: be fast. While your ISR runs, other interrupts of equal or lower priority are held off and no task runs at all. A slow ISR raises the latency of everything else in the system. Microseconds, not milliseconds.

Rule 3: use the `FromISR` functions. FreeRTOS provides two versions of everything that can touch a task: `xQueueSend` and `xQueueSendFromISR`, `xSemaphoreGive` and `xSemaphoreGiveFromISR`. The `FromISR` variants never block, and instead of blocking they report through an output parameter whether they woke a task more important than the one that was interrupted. **Calling the non-ISR version from an ISR is a real bug**, and being able to explain why is a standard interview answer.

```

static void IRAM_ATTR btn_isr(void *arg) {
    uint32_t gpio = (uint32_t)arg;
    BaseType_t hp = pdFALSE;                // "did we wake someone important?"
    xQueueSendFromISR(evt_q, &gpio, &hp);  // non-blocking by construction
}
    
```

```

    if (hp) portYIELD_FROM_ISR();           // switch to them immediately on return
}

```

The `hp` flag, explained properly. It starts `pdFALSE`. If the queue send unblocks a task whose priority is higher than the task that was interrupted, FreeRTOS sets it `pdTRUE`. `portYIELD_FROM_ISR()` then tells the kernel to switch to that task *the moment this ISR returns*, rather than letting the interrupted task resume and waiting for the next scheduler tick. Omit it and everything still works, just with up to one tick (10 ms by default) of extra latency — which is a latency bug that only shows up under load and is exactly the sort of subtle thing a good interviewer digs into.

`IRAM_ATTR`, explained properly. Normally code lives in external flash and is pulled into RAM through a cache as needed. But an interrupt can fire at any instant, *including while the flash is busy* — during a write, or an OTA update, or a cache miss. If the handler's code were sitting in flash at that moment, it would be unreachable and the chip would crash. `IRAM_ATTR` forces the function into internal RAM so it is always available. Any ISR, and anything an ISR calls, needs this. This is the concrete consequence of the memory hierarchy from the "Meet the SoC" chapter.

Debouncing, and why it belongs in the task

A mechanical switch does not close cleanly. The contacts physically bounce for something like 1–20 ms, generating a burst of edges. The hardware faithfully reports every one, so one press can fire the ISR a dozen times.

The fix is to wait for the contacts to settle and then confirm the level. The question is *where*. Not in the ISR — that would mean delaying, which violates Rule 1. So the ISR captures the first edge and the **task** does the waiting: sleep 20 ms, re-read the pin, and only accept it as a real press if the pin is still low. The event is timestamped by the interrupt; the qualification happens where blocking is legal.

This is the **top half / bottom half** split, and it is the single most transferable idea in the lab. Top half: minimal, in interrupt context, captures the event. Bottom half: the real work, in task context, where you may block, allocate, log, and take your time.

TM4C bridge: on the TM4C you debounced with a `delayMs()` inside or right after the handler and it worked because nothing else needed the CPU. Here that would stall the entire system for 20 ms. Same problem, and the RTOS forces the better structure.

Why a queue and not a semaphore here

Lab 7 used a semaphore because the timer had nothing to say beyond "now." Lab 8 uses a **queue** because the ISR has actual *data* to hand over: which GPIO fired. One handler can serve several pins, and the worker needs to know which one. Data crossing a context boundary means queue. Refer back to the decision table in §4.3 — this is that table being used.

The queue also buffers: `xQueueCreate(8, sizeof(uint32_t))` gives eight slots, so a short burst of presses is captured even if the worker is briefly busy. A binary semaphore would have collapsed them into one.

8.4 Procedure & code

```
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/queue.h"
#include "driver/gpio.h"
#include "esp_log.h"

#define BTN_GPIO 0

static QueueHandle_t evt_q;

static void IRAM_ATTR btn_isr(void *arg) {           // interrupt context -- must live in IRAM
    uint32_t gpio = (uint32_t)arg;
    BaseType_t hp = pdFALSE;
    xQueueSendFromISR(evt_q, &gpio, &hp);           // the ONLY safe way to reach a task from an ISR
    if (hp) portYIELD_FROM_ISR();                   // wake the worker immediately if higher priority
}

static void worker_task(void *arg) {
    uint32_t gpio;
    while (1) {
        if (xQueueReceive(evt_q, &gpio, portMAX_DELAY)) {
            vTaskDelay(pdMS_TO_TICKS(20));           // debounce settle -- fine here, forbidden in
the ISR
            if (gpio_get_level(gpio) == 0)           // still low? a real press, not a bounce
                ESP_LOGI("lab8", "button press handled (gpio %lu)", gpio);
        }
    }
}

void app_main(void) {
    gpio_config_t io = {
        .pin_bit_mask = 1ULL << BTN_GPIO,
        .mode          = GPIO_MODE_INPUT,
        .pull_up_en    = GPIO_PULLUP_ENABLE,
        .intr_type     = GPIO_INTR_NEGEDGE,           // falling edge = the moment of press
    };
    gpio_config(&io);
    evt_q = xQueueCreate(8, sizeof(uint32_t));
    xTaskCreate(worker_task, "worker", 3072, NULL, 10, NULL);
    gpio_install_isr_service(0);                     // one service dispatches all per-pin ISRs
    gpio_isr_handler_add(BTN_GPIO, btn_isr, (void *)BTN_GPIO);
}
```

8.5 Code walkthrough, line by line

Forty-one lines. Short, but nearly every line encodes a rule about interrupt context, and this is the lab where getting one of them wrong produces a crash rather than a wrong answer.

Line 3 · `#include "freertos/queue.h"`

Brings in queues: the type `QueueHandle_t` and the create/send/receive functions. New in this lab.

Line 7 · `#define BTN_GPIO 0`

The onboard BOOT button. GPIO0 is also a **strapping pin** — its level at reset tells the chip whether to boot normally or enter the download bootloader, which is what the button is physically for. Reading it after boot is completely safe; driving it as an output is not.

NOTE

If you are on the expansion base, GPIO0 is not there

The 2.54 mm expansion base deliberately omits GPIO0 along with the other strapping pins. The button still works — it is on the devkit — but when §8.7 asks you to put an analyzer probe on it, you will need to hook onto the devkit's own header from above or lift the board off the base. §2.8 covers this.

Line 9 · `static QueueHandle_t evt_q;`

The queue handle. It is at **file scope** and that is not stylistic — it is required. The ISR on line 14 and the task on line 21 are two separate functions in two separate execution contexts, and both need to name the same queue. A local variable in `app_main` would be invisible to both. `static` keeps it private to this file while still being reachable from every function in it.

Like every handle in this series, it is an **opaque token**: the actual queue structure lives in memory the kernel allocated on line 37, and this variable just identifies it.

Line 11 · `static void IRAM_ATTR btn_isr(void *arg)`

`IRAM_ATTR` forces this function into internal RAM instead of leaving it in flash, and the reason is worth understanding properly rather than memorising.

Think of it as **warehouse, shelf, pocket**. Flash is the warehouse: big, slow, and everything lives there permanently. SRAM is the shelf: small and fast. The **cache** is the pocket: the CPU keeps recently-used instructions close at hand, fetching from the warehouse into the pocket as needed. Ordinary code runs perfectly well this way, because a cache miss just means a short pause while the instruction is fetched.

The problem is that an interrupt can fire **at any instant, including while the flash is busy** — during a write, during an OTA update, or while the cache itself is disabled. At that moment, code sitting in flash is simply unreachable, and a handler that tried to execute from there would crash the chip.

`IRAM_ATTR` says "keep this function on the shelf permanently," so it is always available. The cost is a permanent slice of your 512 KB of SRAM, which is why you mark handlers and nothing else.

This applies transitively. Anything the ISR *calls* must also be IRAM-safe, which is a genuinely easy thing to get wrong by calling an innocuous-looking helper.

Line 12 · `uint32_t gpio = (uint32_t)arg;`

Recovers the pin number. Line 40 registered this handler with `(void *)BTN_GPIO` — the number 0 cast to a pointer — and this casts it back. It is a slightly grubby but completely standard C idiom for smuggling a small integer through an API that expects a `void *`. The alternative would be passing a pointer to a real variable, which would then have to outlive the call.

The value of this pattern is that **one handler can serve many pins**: register the same function for GPIO0, GPIO5 and GPIO6 with different `arg` values and it can tell which fired.

Line 13 · `BaseType_t hp = pdFALSE;`

Two FreeRTOS conventions in one line.

`BaseType_t` is FreeRTOS's "natural word size for this architecture" type — 32-bit here, but 16-bit on a small PIC. Using it rather than `int` means the kernel's own code compiles efficiently on every port. You will see it as the return type of most FreeRTOS calls.

`pdFALSE` and `pdTRUE` are FreeRTOS's boolean constants, defined as 0 and 1. The `pd` prefix stands for "portable" and marks them as part of the portability layer, the same prefix as `pdMS_TO_TICKS`. They exist because FreeRTOS predates C99's `<stdbool.h>` and has to compile on compilers that still lack it. Functionally you may treat them as `false` and `true`.

Initialising to `pdFALSE` matters: the next line only ever sets it to `pdTRUE`, never clears it, so it must start false.

Line 14 · `xQueueSendFromISR(evt_q, &gpio, &hp);`

The heart of the lab. FreeRTOS provides **two versions** of everything that can touch a task: `xQueueSend` and `xQueueSendFromISR`, `xSemaphoreGive` and `xSemaphoreGiveFromISR`. You must use the `FromISR` variant here, and the reason is precise.

The normal `xQueueSend` takes a timeout and can **block** — if the queue is full it puts the calling task to sleep until space appears. That is meaningless in an ISR, because *there is no task to put to sleep*. The scheduler cannot suspend an interrupt. Calling the blocking version from interrupt context corrupts the kernel's bookkeeping or crashes outright.

The `FromISR` variant never blocks. If the queue is full it simply returns `errQUEUE_FULL` and the item is dropped. **This listing ignores that return value**, which is worth noticing: if you ever see fewer log lines than button presses, an unchecked `errQUEUE_FULL` is the first place to look.

`&gpio` is passed by address because the queue **copies** the item into its own storage — it does not store your pointer. That is what makes queues safe across contexts: the sender's local variable can go out of scope immediately afterwards and the queued copy is unaffected.

Line 15 · `if (hp) portYIELD_FROM_ISR();`

`hp` is an **output parameter**: the queue call sets it to `pdTRUE` if the send unblocked a task whose priority is higher than the task that was interrupted. The worker runs at priority 10, so in this program it almost always will be.

`portYIELD_FROM_ISR()` then requests a context switch *the moment this ISR returns*, so the woken worker runs immediately rather than waiting for the interrupted task to reach a scheduling point.

Omit it and everything still works — just with up to one full tick, 10 ms by default, of extra latency. That is a latency bug that only shows up under load, and it is exactly the sort of subtlety a good interviewer digs into.

TM4C bridge: on the TM4C the NVIC decided what ran next purely by hardware interrupt priority, and returning from an ISR always resumed the interrupted code. Here an ISR can make a *task* runnable, and this line is you telling the scheduler to act on that immediately. That is the genuinely new mechanism in this lab.

Line 21 · `if (xQueueReceive(evt_q, &gpio, portMAX_DELAY))`

The worker's wait. `xQueueReceive` copies the oldest item into `gpio` and returns `pdTRUE`; with `portMAX_DELAY` it waits forever, so the `if` is effectively always true here (it would matter with a real timeout).

NOTE**Blocking is not polling — they are opposites**

Worth stating plainly because it is easy to get backwards. **Polling** means repeatedly asking "has it happened yet?" in a loop; the task stays **RUNNING** and burns CPU the entire time. **Blocking** means the task is removed from the scheduler's ready list entirely and consumes **zero** CPU until the kernel wakes it. Blocking looks like waiting and is actually the efficient option; polling looks like working and is actually the wasteful one. Lab 1's button loop polled. This blocks.

Line 22 · `vTaskDelay(pdMS_TO_TICKS(20));`

Waits 20 ms for the switch contacts to stop bouncing. **This line is legal here and would be illegal three lines earlier.** Delaying inside the ISR would freeze the entire system for 20 ms; delaying inside a task simply blocks that one task while everything else runs. Same function call, completely different consequence, decided purely by which context you are in. That contrast is the top-half / bottom-half split.

Line 23 · `if (gpio_get_level(gpio) == 0)`

Re-reads the pin after the settle time. Still low means the contacts have settled into a genuine press rather than a bounce transient, so the event is accepted.

WARNING**This debounce has a failure mode worth knowing**

The check asks "is the button *still* held 20 ms later." That correctly rejects bounce, but it also means a **very fast tap** — pressed and released inside 20 ms — is discarded as if it were noise, because the pin has already returned high by the time you look.

And if you **hold** the button down, only the first falling edge produces an interrupt, so you get exactly one log line no matter how long you hold. That is usually what you want for a button, but it is a design decision, not a law. If you needed press-and-hold behaviour you would trigger on both edges and time the interval yourself.

A cleaner alternative the S3 offers in hardware: each GPIO has a **glitch filter** that discards pulses shorter than a configurable width, filtering bounce before it ever reaches your ISR. Look up `gpio_glitch_filter` in the GPIO documentation — using hardware to solve this instead of a software delay is a genuinely better answer if it comes up.

Line 24 · `ESP_LOGI("lab8", "button press handled (gpio %lu)", gpio);`

`%lu` is the conversion for an `unsigned long`, matching `uint32_t`. Note the tag is a literal string here rather than a file-scope `TAG` constant — both are common; one `TAG` per file is the tidier habit.

Logging is fine here for the same reason the delay is: task context. `ESP_LOGI` from inside the ISR would be a serious bug, and §8.7 asks you to try it deliberately so you see what happens.

Line 31 · `.pin_bit_mask = 1ULL << BTN_GPIO`

A 64-bit mask with one bit set per pin being configured. The `ULL` makes the literal 64-bit, which is necessary because this chip has pins above 31 and `1 << 40` on a 32-bit `int` is undefined behaviour.

Line 34 · `.intr_type = GPIO_INTR_NEGEDGE`

The one field that makes this an interrupt rather than a plain input. **NEGEDGE** fires on a high-to-low transition — the exact instant the pull-up-held pin is dragged to ground by the press. The alternatives are **POSEDGE** (release), **ANYEDGE** (both), and the two level-triggered types, which fire continuously while the pin sits at a level and are used for things like a sensor's "data ready" line.

Line 36 · `gpio_config(&io);`

Applies the whole struct, including arming the interrupt. **Note carefully where this line sits relative to line 40.**

WARNING

The ordering bug that will bite you

This configuration *enables the interrupt*, but the handler is not registered until line 40. In this listing the gap is only a few microseconds of setup and the button is not being pressed, so nothing goes wrong.

It is still the wrong order in principle, and it becomes a real bug the moment the interrupt source is something that can fire on its own — a free-running sensor, an external clock. An interrupt arriving in that window has no handler to dispatch to. **The safe order is: create the queue, create the task, install the ISR service, register the handler, and enable the interrupt last.** You can get that by leaving `.intr_type` out of the config struct and calling `gpio_set_intr_type()` after `gpio_isr_handler_add()`. Worth restructuring in your own copy — and worth mentioning in an interview, because noticing initialisation-order hazards is a senior instinct.

Line 37 · `evt_q = xQueueCreate(8, sizeof(uint32_t));`

Allocates the queue: **8 slots**, each `sizeof(uint32_t)` = 4 bytes, so 32 bytes of item storage plus kernel overhead. Both arguments are decisions. The item size must match what you send; the depth is your burst tolerance — eight presses can queue up while the worker is busy, and the ninth is dropped. A depth of 1 would collapse rapid presses into one.

This must happen before line 40, because the ISR uses `evt_q` and an unregistered handler firing against a null queue handle is a crash.

Line 38 · `xTaskCreate(worker_task, "worker", 3072, NULL, 10, NULL);`

Priority **10** is deliberately high so the worker preempts promptly when the ISR wakes it — which is also what makes `hp` on line 13 come back `pdTRUE`. The 3072-byte stack is comfortable for a task that logs.

Line 39 · `gpio_install_isr_service(0);`

Starts the **shared GPIO interrupt dispatcher**. The hardware has one interrupt line for the whole GPIO peripheral, not one per pin, so ESP-IDF installs a single low-level handler that reads which pin fired and calls the right per-pin function. The `0` argument is interrupt allocation flags; zero means defaults.

Call it **once** for the whole program. Calling it twice returns `ESP_ERR_INVALID_STATE`, which is a confusing error if you have wrapped it in `ESP_ERROR_CHECK` and forgotten it was already installed.

Line 40 · `gpio_isr_handler_add(BTN_GPIO, btn_isr, (void *)BTN_GPIO);`

Registers your handler for this specific pin, with the third argument becoming the `arg` that line 12 casts back. Three arguments: which pin, which function, what to pass it.

TM4C bridge: lines 39 and 40 together replace enabling the pin's interrupt in the GPIO registers and enabling its line in the NVIC. The "install one service, then add a handler per pin" split has no direct TM4C equivalent, because there you wrote the single `GPIOPortA_Handler` yourself and worked out inside it which pin had fired.

KEY POINT

A debugging function worth knowing

`gpio_dump_io_configuration(stdout, 1ULL << BTN_GPIO)` prints a pin's actual live configuration — pull-ups, direction, interrupt type, peripheral routing. When a pin is not behaving and you are certain your config struct is right, this tells you what the hardware *actually* ended up with rather than what you believe you asked for.

8.6 Docs & references

DOCS

Trace every line to its source

`gpio_config_t.intr_type`, `gpio_install_isr_service`, `gpio_isr_handler_add` — Peripherals → **GPIO**. Read the full list of trigger types and choose deliberately rather than copying `NEGEDGE`.

`xQueueCreate`, `xQueueReceive`, `xQueueSendFromISR`, `portYIELD_FROM_ISR` — FreeRTOS API. **Read the `pxHigherPriorityTaskWoken` parameter description in full**; that parameter is precisely what interviewers probe.

`IRAM_ATTR` and IRAM rules — Programming Guide → API Guides → **Memory Types**, plus the "IRAM-Safe Interrupt Handlers" discussion.

8.7 Verify, troubleshoot, your turn

Verify. One press produces exactly one log line. Press rapidly and each press still registers once, because the queue buffers a short burst even if the worker is briefly busy.

Troubleshoot

Symptom	Cause and fix
Crash the instant you press	Missing <code>IRAM_ATTR</code> , or a non- <code>FromISR</code> function called inside the handler.
Several log lines per press	The debounce check was removed, or the settle time is too short for your switch.
Nothing happens on press	The ISR service was not installed, or the trigger edge is wrong.
Log appears late	Missing <code>portYIELD_FROM_ISR</code> , so the worker waits for the next tick.

Your turn

1. Change to `GPIO_INTR_ANYEDGE` and log both press and release, using the pin level to tell them apart. You now have to reason about which edge produced which event.
2. Remove the debounce and count raw interrupts per press. That count is physical evidence of contact bounce and makes a concrete war story.
3. Deliberately call `ESP_LOGI` from inside the ISR and observe what happens. Understanding *why* that is illegal, from having seen it fail, is worth more than being told.

Errata and corrections

Defects confirmed against the hardware and the vendor documentation, with the evidence that established each one. They are printed rather than quietly patched because how an error was caught is usually more instructive than the error.

Additional corrections

Overbroad claim. The code comment "the ONLY safe way to reach a task from an ISR" is wrong. Direct-to-task notifications, the ISR-safe semaphore and event-group APIs, and the other `FromISR` functions are all valid. The manual contradicts itself here: Lab 20 uses `vTaskNotifyGiveFromISR` for exactly this purpose. The correct rule is **"use only APIs documented as ISR-safe; a queue is one option, chosen here because it carries data."** Also, `IRAM_ATTR` on the handler is necessary but not sufficient for cache safety — interrupt allocation flags, every transitively called function, and the data touched all matter.

The series at a glance

Each lab is a complete, independently buildable ESP-IDF project. This is Lab 8.

#	Lab	Phase
00	Toolchain: hello, ESP32-S3	Foundations
01	GPIO: RGB LED and BOOT button	Foundations
02	FreeRTOS tasks and drift-free timing	Foundations
03	UART and the loopback test	Foundations
04	Hardware timers and the 1 kHz problem	Foundations
05	I ² C and the IMU bring-up	Foundations
06	PWM with the LEDC peripheral	Foundations
07	The sensor pipeline (capstone)	Foundations
08 ←	GPIO interrupts and ISR-safe code	Past the coursework
09	SPI, full-duplex and fast	Past the coursework
10	NVS: data that survives power loss	Past the coursework
11	Unit testing with Unity	Past the coursework
12	Deep sleep and waking up	Past the coursework
13	Wi-Fi station and JSON telemetry	Past the coursework
14	BLE GATT with NimBLE	Past the coursework
15	OTA and the rollback safety net	Past the coursework
16	Priority inversion and mutexes	Past the coursework
17	CAN (TWAI): broadcast by message ID	Past the coursework
18	PID, anti-windup and derivative kick	Past the coursework
19	The ML inference harness	Gap-closers
20	Continuous ADC: DMA-backed acquisition	Gap-closers
21	On-chip debugging: JTAG, OpenOCD, GDB	Gap-closers
22	A real display (SPI TFT + DMA)	Gap-closers
23	Production test mode	Gap-closers

Written against ESP-IDF v5.5 on an ESP32-S3 N16R8 module. Pin assignments are choices, not constraints — the ESP32-S3 GPIO matrix routes almost any peripheral to almost any pin, with the exception of GPIO26–GPIO37, which are committed to the on-board flash and PSRAM and must never be used.